



HNDIT 3052 Operating Systems

Threads



Threads

Topic: Threads

Subtopics:

- The Thread Model
- POSIX Threads
- Implementation of Thread



Threads

- **What is a Thread?**
- Let us take an example of a human body. A human body has different parts having different functionalities which are working parallelly (Eg: *Eyes, ears, hands*, etc). Similarly in computers, a single process might have multiple functionalities running parallelly where each functionality can be considered as a thread.

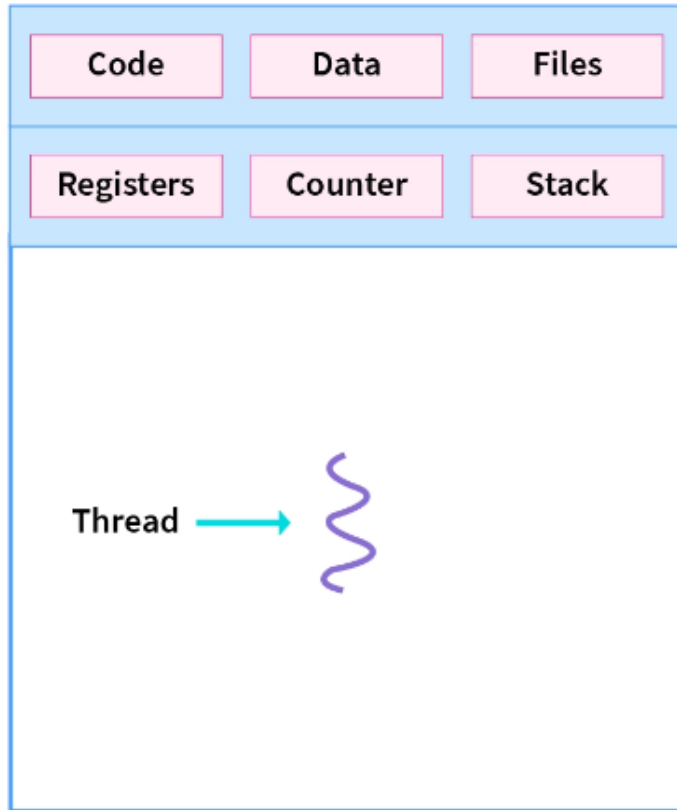


What is Thread in OS?

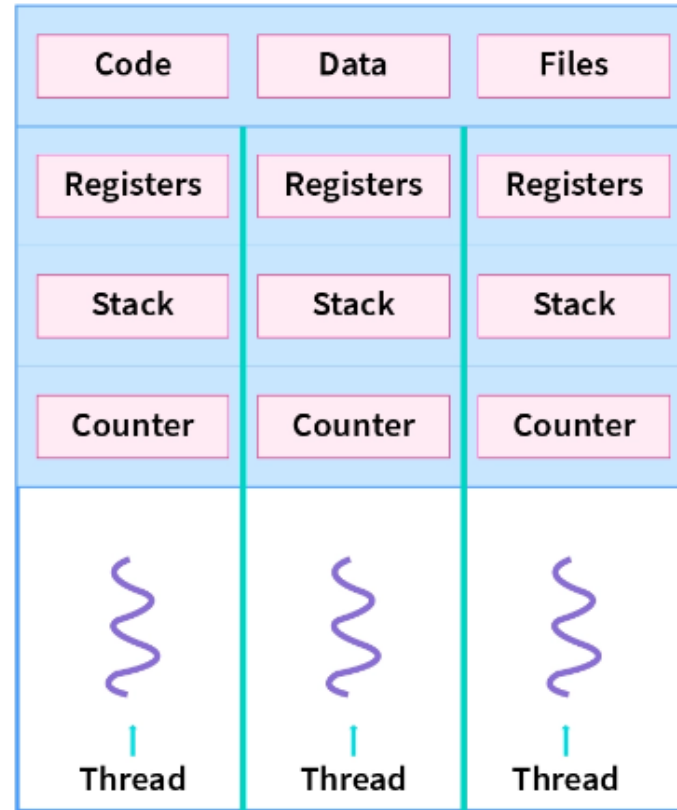
- **Thread is a sequential flow of tasks within a process.** Threads in OS can be of the same or different types. Threads are used to increase the performance of the applications.
- Each thread has its own program counter, stack, and set of registers. But the threads of a single process might share the same code and data/file. **Threads are also termed as lightweight processes as they share common resources.**
- **Eg:** While playing a movie on a device the audio and video are controlled by different threads in the background.



Eg: While playing a movie on a device the audio and video are controlled by different threads in the background.



Single-threaded process



Multithreaded process

The above diagram shows the difference between a single-threaded process and a multithreaded process and the resources that are shared among threads in a multithreaded process.



Components of Thread

A thread has the following three components:

1. Program Counter
2. Register Set
3. Stack space



Why do we need Threads?

Threads in the operating system provide multiple benefits and improve the overall performance of the system. Some of the reasons threads are needed in the operating system are:

- Since threads use the same data and code, the operational cost between threads is low.
- Creating and terminating a thread is faster compared to creating or terminating a process.
- **Context switching** is faster in threads compared to processes.



Context Switching:

- A context switch is a procedure that a computer's **CPU** (central processing unit) follows to change from one **task** (or process) to another while ensuring that the tasks do not conflict.
- Effective context switching is critical if a computer is to provide user-friendly multitasking.
- In a CPU, the term "context" refers to the data in the **registers** and program counter at a specific moment in time. A register holds the current CPU **instruction**. A program counter, also known as an instruction address register, is a small amount of fast **memory** that holds the address of the instruction to be executed immediately after the current one.



Context Switching: ...

- A context switch can be performed entirely in hardware (physical media). Older CPUs, such as those in the x86 series, do it that way.
- However, most modern CPUs perform context switches by means of software (programming). A modern CPU can perform hundreds of context switches per second.
- Therefore, the user gets the impression that the computer is performing multiple tasks in a parallel fashion, when the CPU actually alternates or rotates between or among the tasks at a high rate of speed.



Why Multithreading?

In Multithreading, the idea is to divide a single process into multiple threads instead of creating a whole new process. Multithreading is done to achieve parallelism and to improve the performance of the applications as it is faster in many ways which were discussed above. The other advantages of multithreading are mentioned below.

- **Resource Sharing:** Threads of a single process share the same resources such as code, data/file.
- **Responsiveness:** Program responsiveness enables a program to run even if part of the program is blocked or executing a lengthy operation. Thus, increasing the responsiveness to the user.
- **Economy:** It is more economical to use threads as they share the resources of a single process. On the other hand, creating processes is expensive.



Process vs Thread

- **Process simply means any program in execution while the thread is a segment of a process.** The main differences between process and thread are mentioned below:



Process	Thread
Processes use more resources and hence they are termed as heavyweight processes.	Threads share resources and hence they are termed as lightweight processes.
Creation and termination times of processes are slower.	Creation and termination times of threads are faster compared to processes.
Processes have their own code and data/file.	Threads share code and data/file within a process.
Communication between processes is slower.	Communication between threads is faster.
Context Switching in processes is slower.	Context switching in threads is faster.



Process vs. Threads...

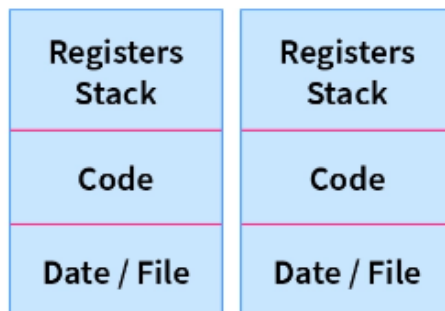
Processes are independent of each other.

Threads, on the other hand, are interdependent. (i.e they can read, write or change another thread's data)

Eg: Opening two different browsers.

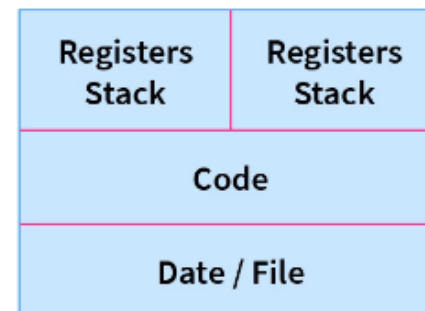
Eg: Opening two tabs in the same browser.

PROCESS



Two Different Process

THREAD



Two Threads of a single process

The above diagram shows how the resources are shared in two different processes vs two threads in a single process.



Another Different comparison of Thread & Process.

S.NO	Process	Thread
1.	Process means any program is in execution.	Thread means a segment of a process.
2.	The process takes more time to terminate.	The thread takes less time to terminate.
3.	It takes more time for creation.	It takes less time for creation.
4.	It also takes more time for context switching.	It takes less time for context switching.
5.	The process is less efficient in terms of communication.	Thread is more efficient in terms of communication.



S.NO	Process	Thread
6.	Multiprogramming holds the concepts of multi-process.	We don't need multi programs in action for multiple threads because a single process consists of multiple threads.
7.	The process is isolated.	Threads share memory.
8.	The process is called the heavyweight process.	A Thread is lightweight as each thread in a process shares code, data, and resources.
9.	Process switching uses an interface in an operating system.	Thread switching does not require calling an operating system and causes an interrupt to the kernel.
10.	If one process is blocked then it will not affect the execution of other processes	If a user-level thread is blocked, then all other user-level threads are blocked.



S.NO	Process	Thread
11.	The process has its own Process Control Block, Stack, and Address Space.	Thread has Parents' PCB, its own Thread Control Block, and Stack and common Address space.
12.	Changes to the parent process do not affect child processes.	Since all threads of the same process share address space and other resources so any changes to the main thread may affect the behavior of the other threads of the process.
13.	A system call is involved in it.	No system call is involved, it is created using APIs.
14.	The process does not share data with each other.	Threads share data with each other.

Note: In some cases where the thread is processing a bigger workload compared to a process's workload then the thread may take more time to terminate. But this is an extremely rare situation and has fewer chances to occur.



Types of Thread

- **1. User Level Thread:**
- **2. Kernel level Thread:**



User Threads and Kernel Threads

1. **User threads** - management done by user-level threads library.
 - Three primary thread libraries:
 - POSIX **Pthreads**
 - Windows threads
 - Java threads
2. **Kernel threads** - Supported by the Kernel
 - Examples – virtually all general purpose operating systems, including:
 - Windows
 - Solaris
 - Linux
 - Tru64 UNIX
 - Mac OS X



1. User Level Thread:

User-level threads are implemented and managed by the user and the kernel is not aware of it.

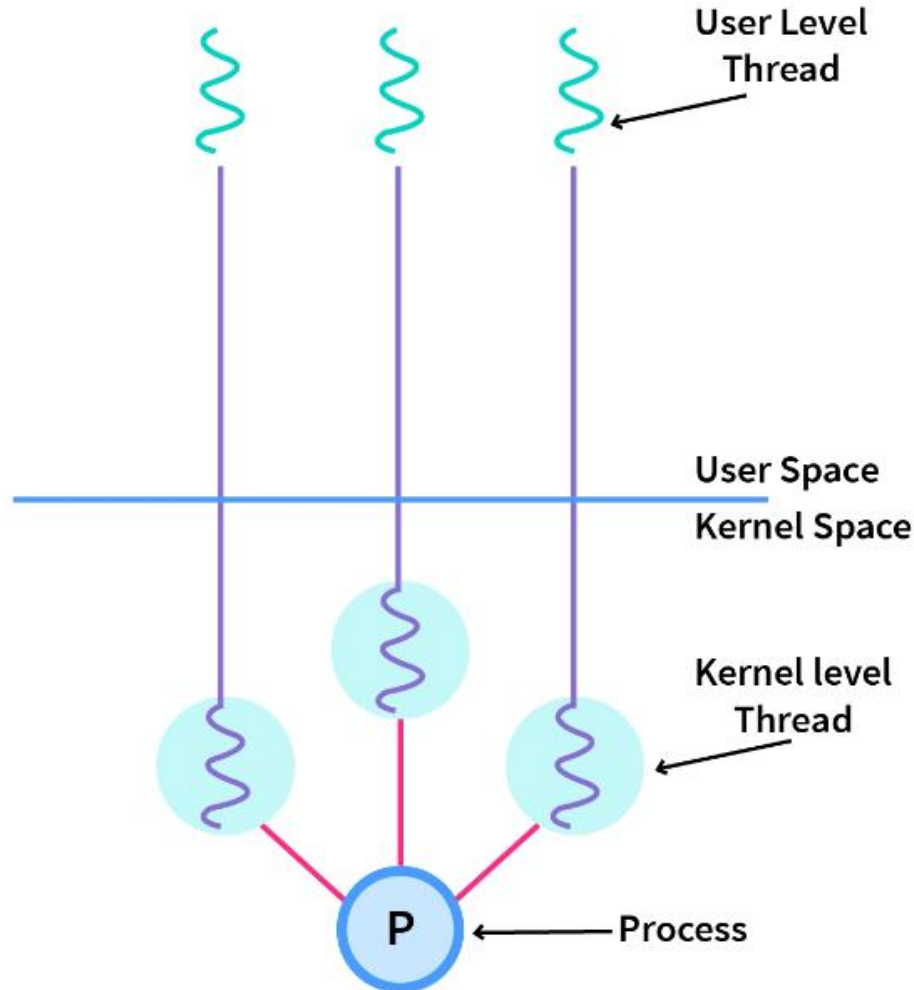
- User-level threads are **implemented using user-level libraries and the OS does not recognize these threads.**
- User-level thread is **faster to create and manage compared to kernel-level thread.**
- **Context switching in user-level threads is faster.**
- If one user-level thread performs a blocking operation then the entire process gets blocked.
 - Eg: POSIX threads, Java threads, etc.



2. Kernel level Thread:

Kernel level threads are implemented and managed by the OS.

- **Kernel level threads are implemented using system calls and Kernel level threads are recognized by the OS.**
- **Kernel-level threads are slower to create and manage compared to user-level threads.**
- **Context switching in a kernel-level thread is slower.**
- **Even if one kernel-level thread performs a blocking operation, it does not affect other threads. Eg: Window Solaris.**



The above diagram shows the functioning of user-level threads in **user space** and kernel-level threads in **kernel space**.



Advantages of Threading:

1. Threads improve the overall performance of a program.
2. Threads increases the responsiveness of the program
3. Context Switching time in threads is faster.
4. Threads share the same memory and resources within a process.
5. Communication is faster in threads.
6. Threads provide concurrency within a process.
7. Enhanced throughput of the system.
8. Since different threads can run parallelly, threading enables the utilization of the multiprocessor architecture to a greater extent and increases efficiency.



Issues with Threading

There are a number of issues that arise with threading. Some of them are mentioned below:

- **The semantics of fork() and exec() system calls: The fork() call is used to create a duplicate child process.**
- **During a fork() call the issue that arises is whether the whole process should be duplicated or just the thread which made the fork() call should be duplicated.**
- **The exec() call replaces the whole process that called it including all the threads in the process with a new program.**



Issues with Threading...

- **Thread cancellation:** The termination of a thread before its completion is called thread cancellation and the terminated thread is termed as target thread. Thread cancellation is of two types:
 - **Asynchronous Cancellation:** In asynchronous cancellation, one thread immediately terminates the target thread.
 - **Deferred Cancellation:** In deferred cancellation, the target thread periodically checks if it should be terminated.



Issues with Threading...

- **Signal handling:** In UNIX systems, a signal is used to notify a process that a particular event has happened. Based on the source of the signal, signal handling can be categorized as:
 - **Asynchronous Signal:** The signal which is generated outside the process which receives it.
 - **Synchronous Signal:** The signal which is generated and delivered in the same process.



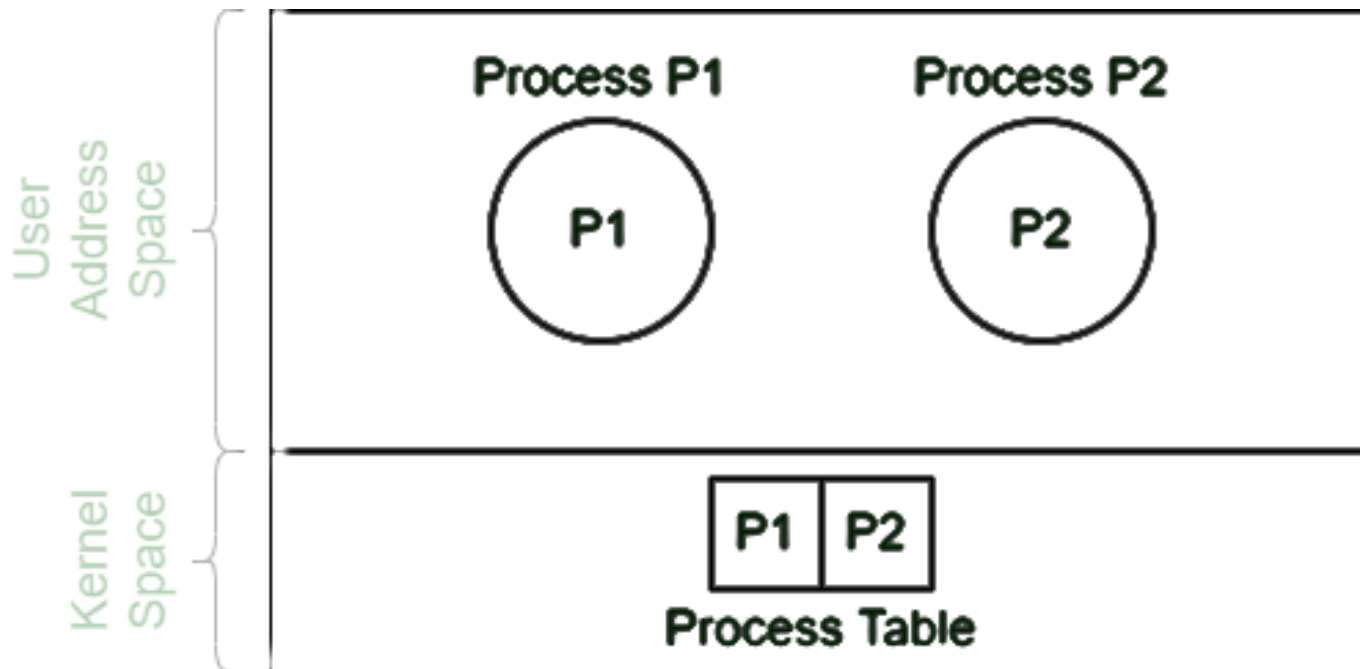
The Thread Model

Thread Models in Operating System:

- A thread is a light weight process which is similar to a process where every process can have one or more threads. Each thread contains a Stack and a Thread Control Block. There are four basic thread models :
 1. User Level Single Thread Model :
 2. User Level Multi Thread Model :
 3. Kernel Level Single Thread Model :
 4. Kernel Level Multi Thread Model :

1. User Level Single Thread Model :

- Each process contains a single thread.
- Single process is itself a single thread.
- process table contains an entry for every process by maintaining its PCB.

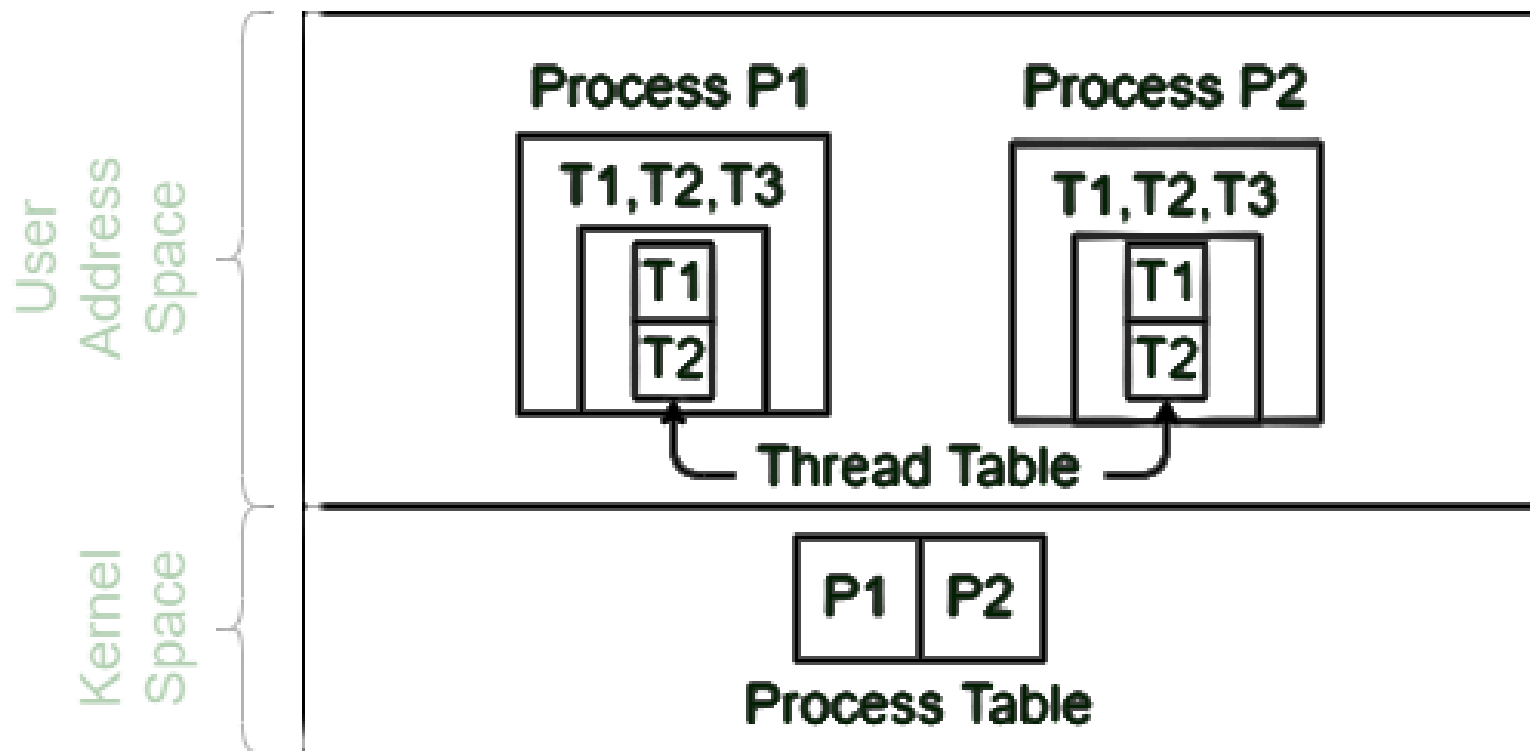




2. User Level Multi Thread Model :

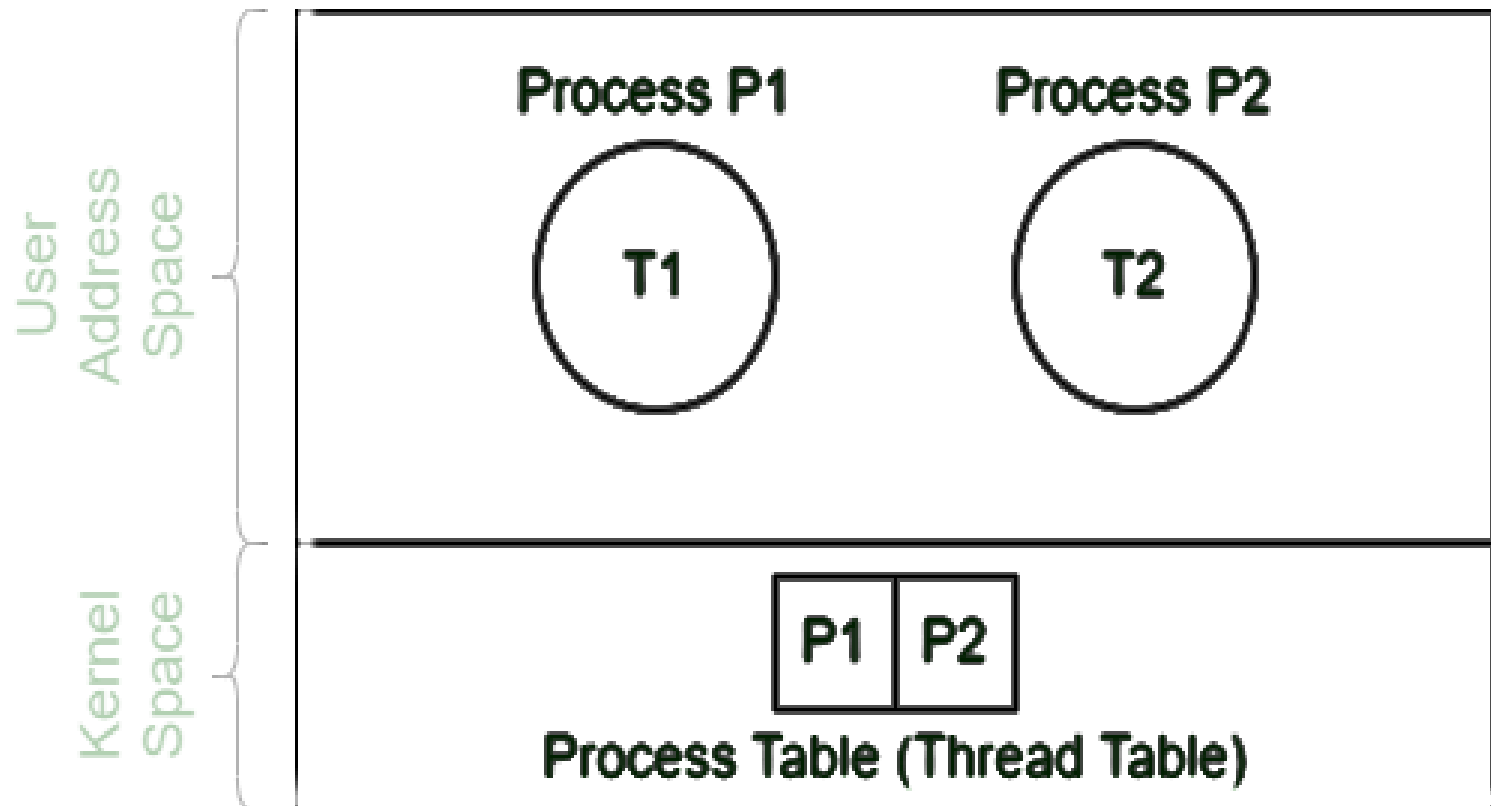
- Each process contains multiple threads.
- All threads of the process are scheduled by a thread library at user level.
- Thread switching can be done faster than process switching.
- Thread switching is independent of operating system which can be done within a process.
- Blocking one thread makes blocking of entire process.
- Thread table maintains Thread Control Block of each thread of a process.
- Thread scheduling happens within a process and not known to Kernel.

2. User Level Multi Thread Model ...



3. Kernel Level Single Thread Model :

- Each process contains a single thread.
- Thread used here is kernel level thread.
- Process table works as thread table.

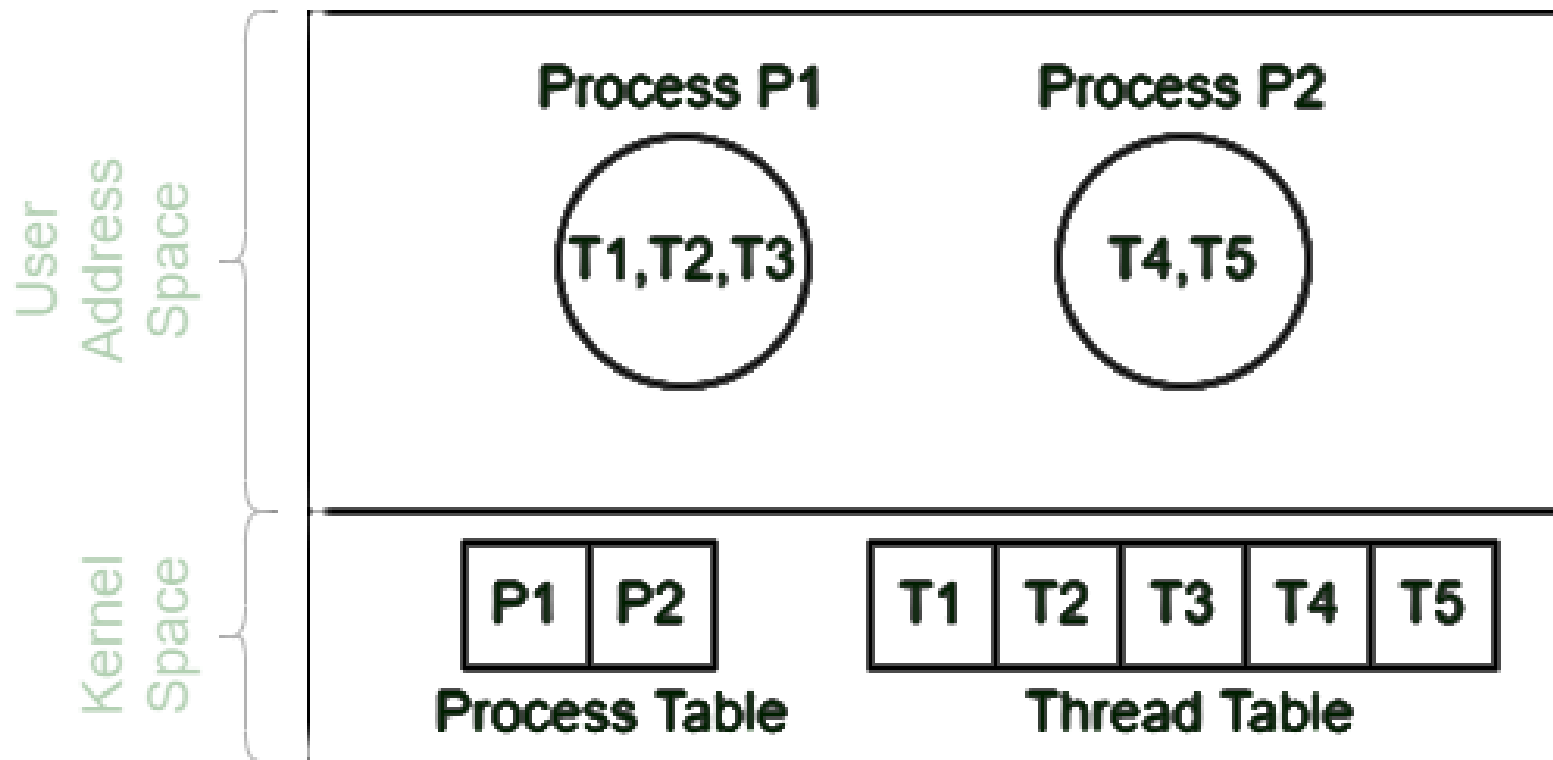




4. Kernel Level Multi Thread Model :

- Thread scheduling is done at kernel level.
- Fine grain scheduling is done on a thread basis.
- If a thread blocks, another thread can be scheduled without blocking the whole process.
- Thread scheduling at Kernel process is slower compared to user level thread scheduling.
- Thread switching involves switch.

4. Kernel Level Multi Thread Model ...



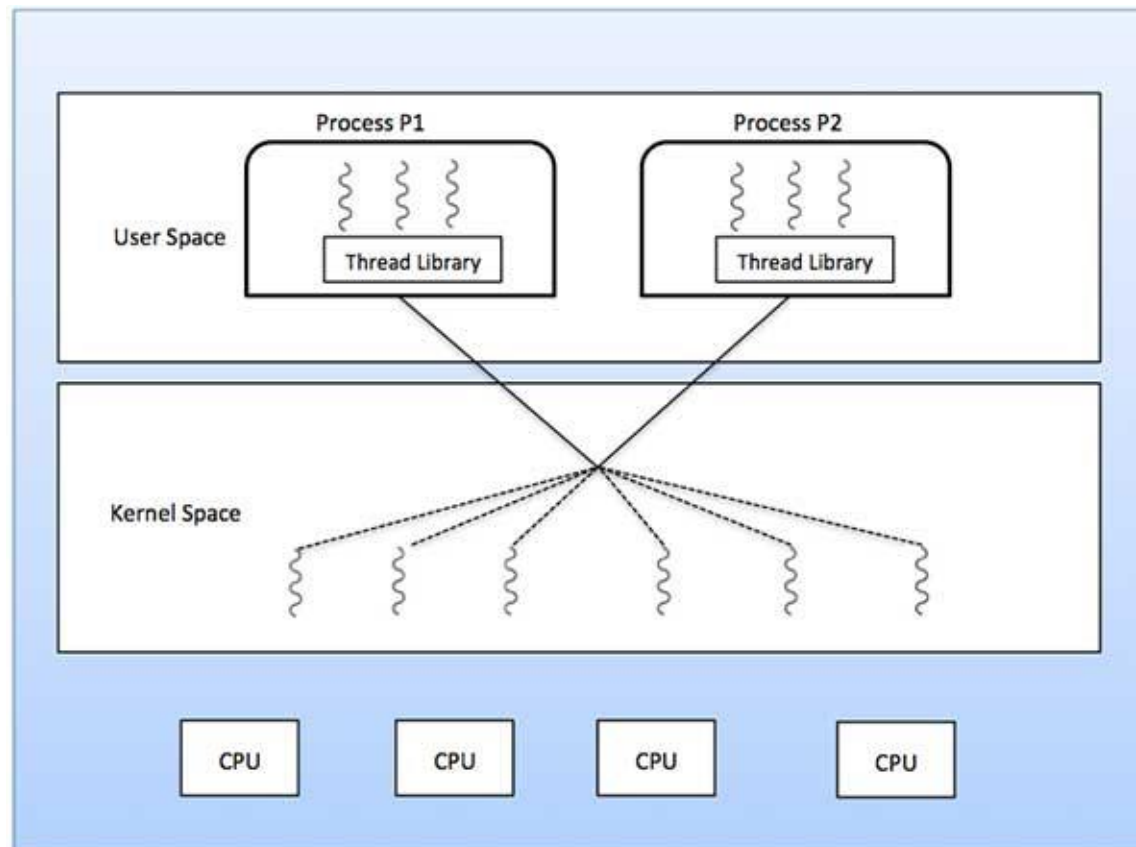


Multithreading Models

- Some operating system provide a combined user level thread and Kernel level thread facility. Solaris is a good example of this combined approach. In a combined system, multiple threads within the same application can run in parallel on multiple processors and a blocking system call need not block the entire process. Multithreading models are three types.
 1. Many to many relationship.
 2. Many to one relationship.
 3. One to one relationship.

i. Many to Many Multithreading Model:

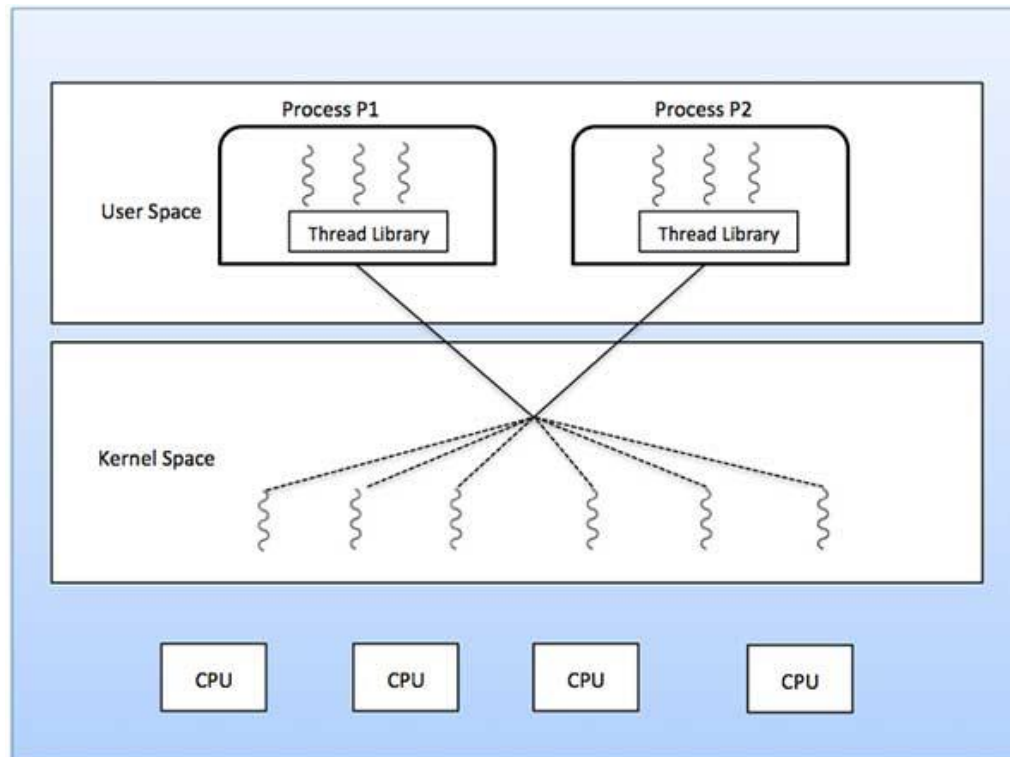
- The many-to-many model **multiplexes** any number of user threads onto an equal or smaller number of kernel threads.





i. Many to Many **Multithreading** Model...

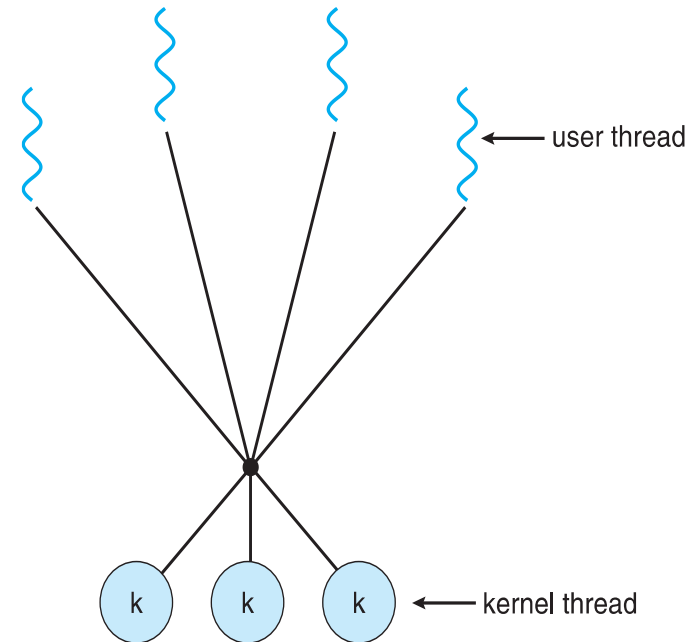
- The following diagram shows the many-to-many threading model where 6 user level threads are multiplexing with 6 kernel level threads. In this model, developers can create as many user threads as necessary and the corresponding Kernel threads **can run in parallel on a multiprocessor machine**. This model **provides the best accuracy on concurrency** and when a **thread** performs a **blocking system call**, the **kernel** can **schedule another thread for execution**.





i. Many to Many **Multithreading** Model...

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Solaris prior to version 9
- Windows with the *ThreadFiber* package.
- **Fiber:**
 - A *fiber* is a unit of execution that must be manually scheduled by the application.
 - Fibers run in the context of the threads that schedule them.
 - Each thread can schedule multiple fibers. In general, fibers do not provide advantages over a well-designed multithreaded application.
 - However, using fibers can make it easier to port applications that were designed to schedule their own threads.





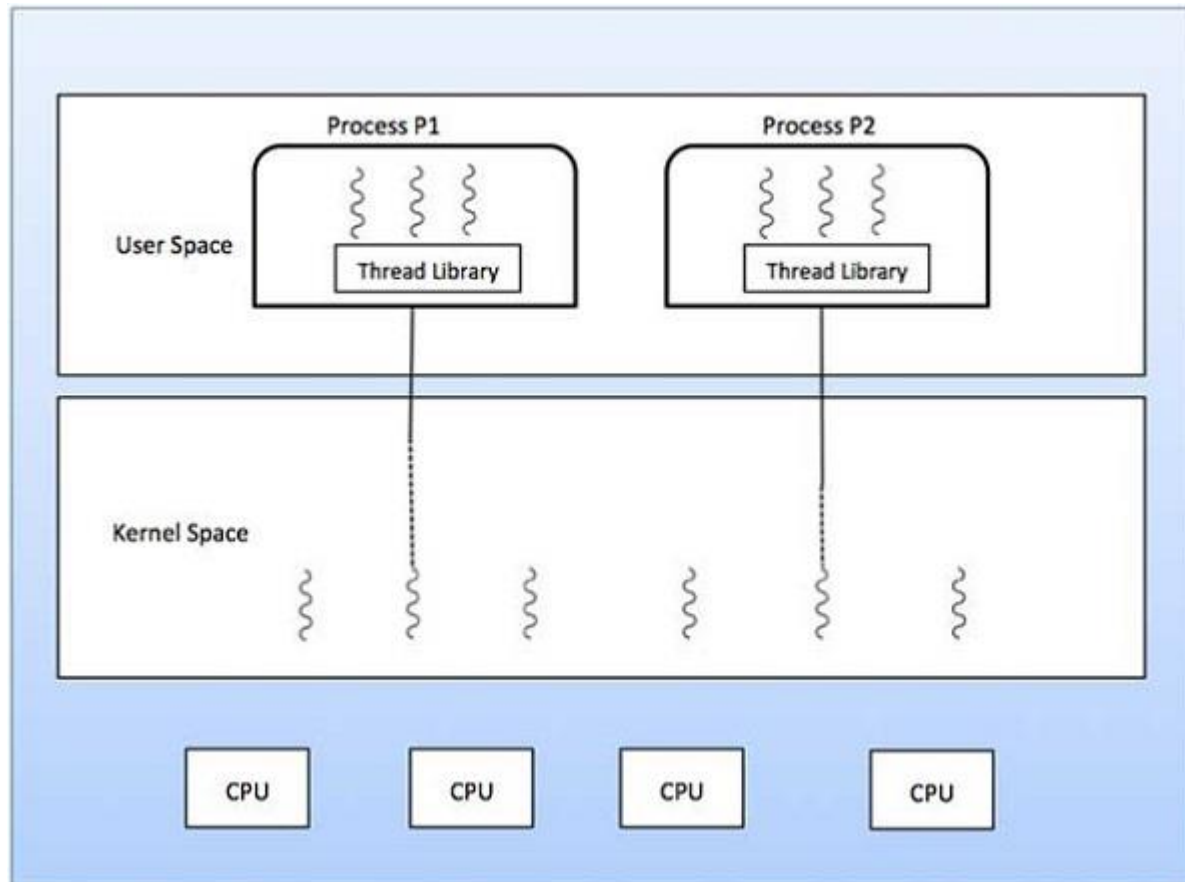
ii. Many to One **Multithreading** Model

- Many-to-one model maps many user level threads to one Kernel-level thread. Thread management is done in user space by the thread library. When thread makes a blocking system call, the entire process will be blocked. Only one thread can access the Kernel at a time, so multiple threads are unable to run in parallel on multiprocessors.



ii. Many to One **Multithreading** Model...

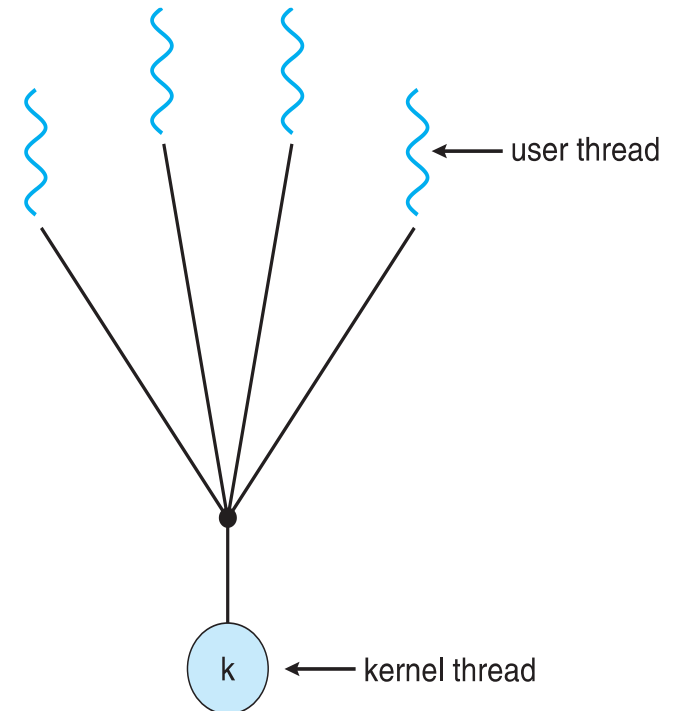
- If the user-level thread libraries are implemented in the operating system in such a way that the system does not support them, then the Kernel threads use the many-to-one relationship modes.





ii. Many to One Multithreading Model...

- ❑ Many user-level threads mapped to single kernel thread.
- ❑ One thread blocking causes all to block.
- ❑ Multiple threads may not run in parallel on multicore system because only one may be in kernel at a time.
- ❑ Few systems currently use this model.
- ❑ Examples:
 - ❑ **Solaris Green Threads**
 - ❑ **GNU Portable Threads**





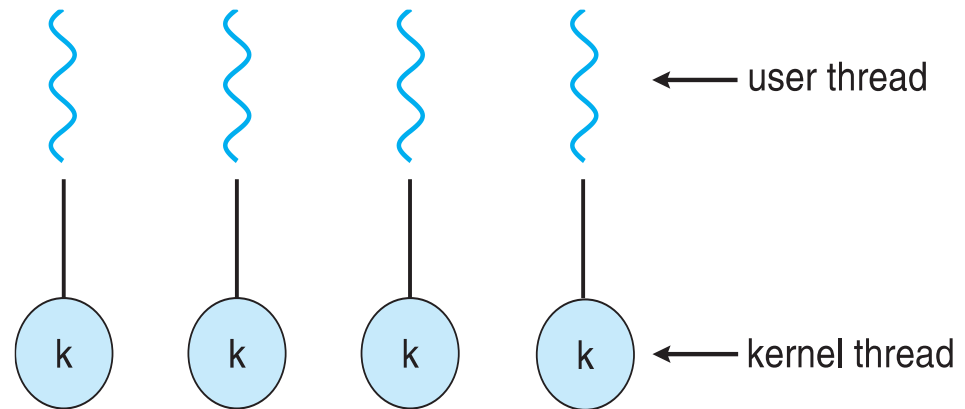
iii. One to One **Multithreading** Model

- There is one-to-one relationship of user-level thread to the kernel-level thread. This model provides more concurrency than the many-to-one model. It also allows another thread to run when a thread makes a blocking system call. It supports multiple threads to execute in parallel on microprocessors.



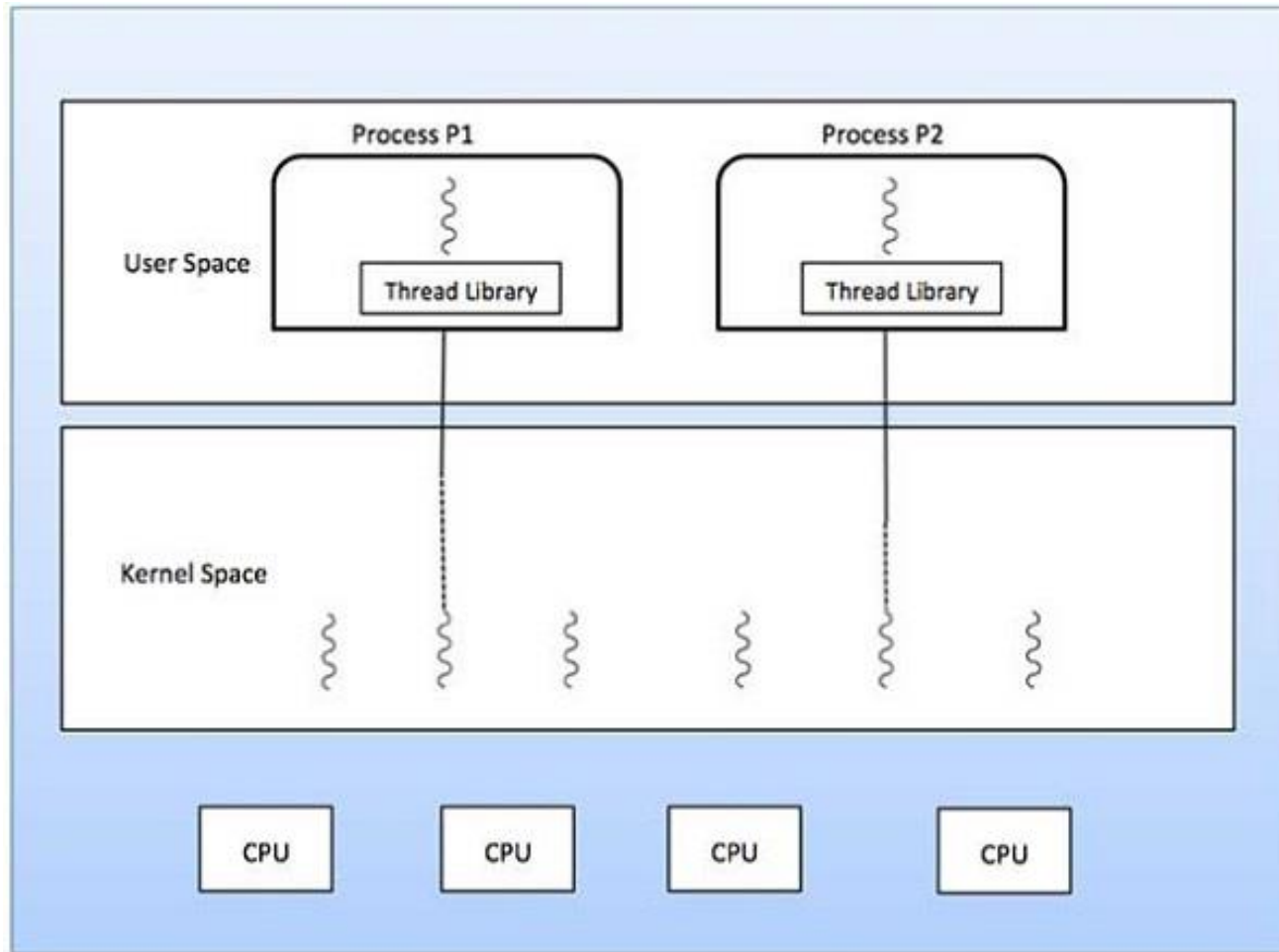
iii. One to One Multithreading Model...

- ❑ Each user-level thread maps to kernel thread
- ❑ Creating a user-level thread creates a kernel thread
- ❑ More concurrency than many-to-one
- ❑ Number of threads per process sometimes restricted due to overhead
- ❑ Examples
 - ❑ Windows
 - ❑ Linux
 - ❑ Solaris 9 and later



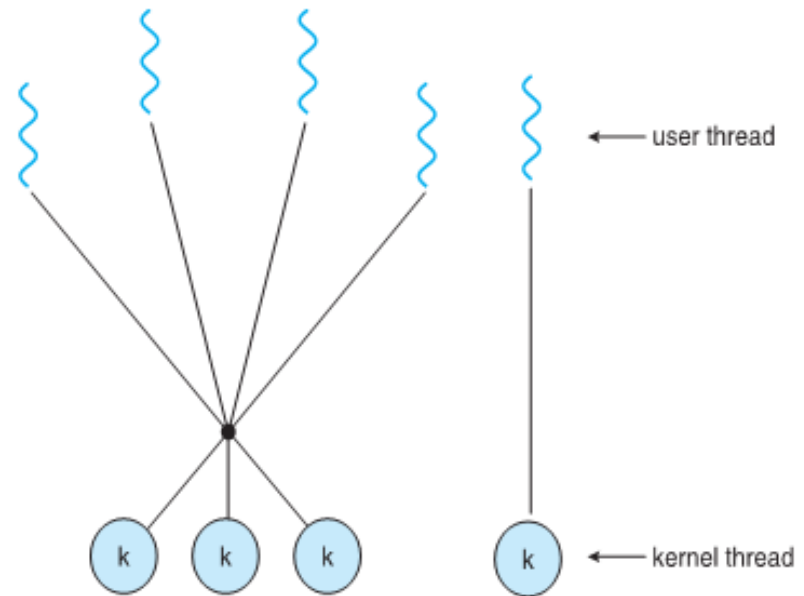
iii. One to One **Multithreading** Model...

- Disadvantage of this model is that creating user thread requires the corresponding Kernel thread. OS/2, windows NT and windows 2000 use one to one relationship model.



(iv). **Two-level Model** (similar to many to many model, but:)

- Similar to M:M, except that **it allows a user thread to be bound to kernel thread**
- Examples
 - IRIX
 - HP-UX
 - Tru64 UNIX
 - Solaris 8 and earlier





How multiprogramming of processes works?

- When a multithreaded process is run on a single-CPU system, the threads take turns running.
- In Fig. 2-1 (next slide) , we saw how multiprogramming of processes works.
- By switching back and forth among **multiple processes**, the system gives the illusion of separate sequential processes running in parallel.
- Multithreading works the same way. The CPU switches rapidly back and forth among the threads, providing the illusion that the threads are running in parallel, albeit on a slower CPU than the real one. With three compute-bound threads in a process, the threads would appear to be running in parallel, each one on a CPU with one-third the speed of the real CPU.

How multiprogramming of processes works?

- Different threads in a process are not as independent as different processes. All threads have exactly the same address space, which means that they also share the same global variables. Since every thread can access every memory address within the process' address space, one thread can read, write, or even wipe out another thread's stack.
- There is no protection between threads because
 - (1) it is impossible, and
 - (2) it should not be necessary.

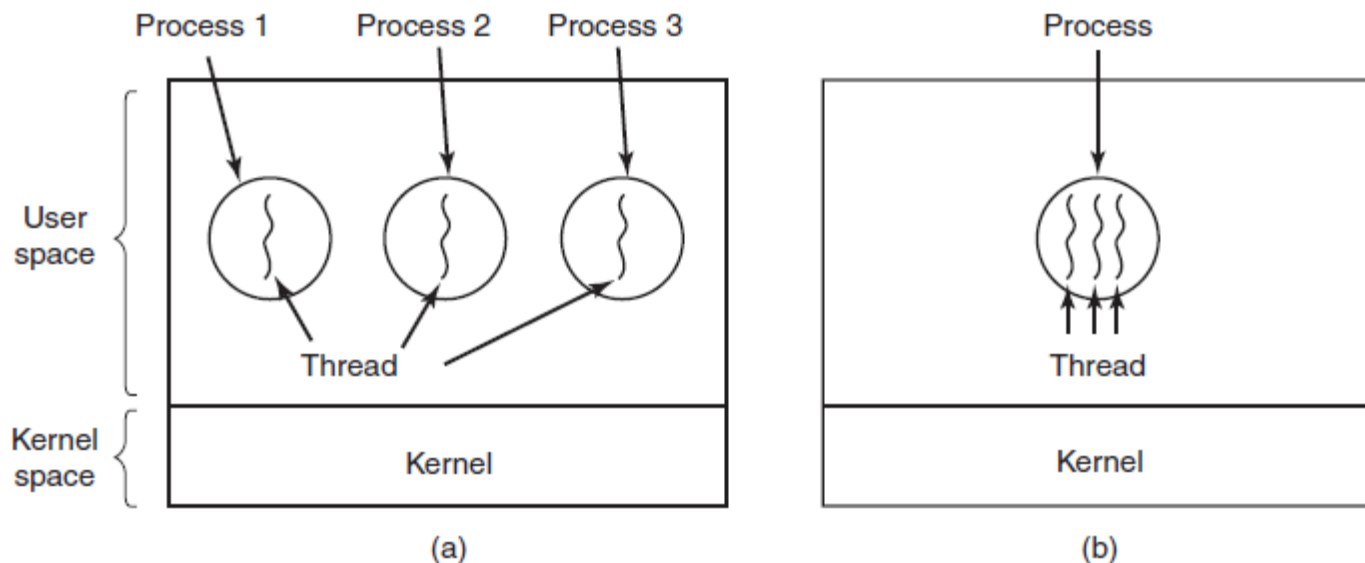


Figure 2-11. (a) Three processes each with one thread. (b) One process with three threads.



Each thread has its own stack. Give the reason.

- **Each thread** has its **own stack**, as illustrated in Fig. 2-13 in next slide.
- **Each thread's stack** contains **one frame** for each procedure called but not yet returned from.
- **This frame contains** the **procedure's local variables** and the **return address** to use when the procedure call has finished.
- **Example:**
 - If procedure X calls procedure Y and,
 - Y calls procedure Z, then
 - while Z is executing,
 - the frames for X, Y, and Z will all be on the stack.
- **Each thread** will generally call **different procedures** and thus have a different execution history.
- This is why each thread needs its own stack.



Each thread has its own stack. Give the reason...

- **Example:**

- If procedure X calls procedure Y and,
- Y calls procedure Z, then
- while Z is executing,
- the frames for X, Y, and Z will all be on the stack.

- **Each thread** will generally call **different procedures** and thus have a different execution history.
- This is why each thread needs its own stack.

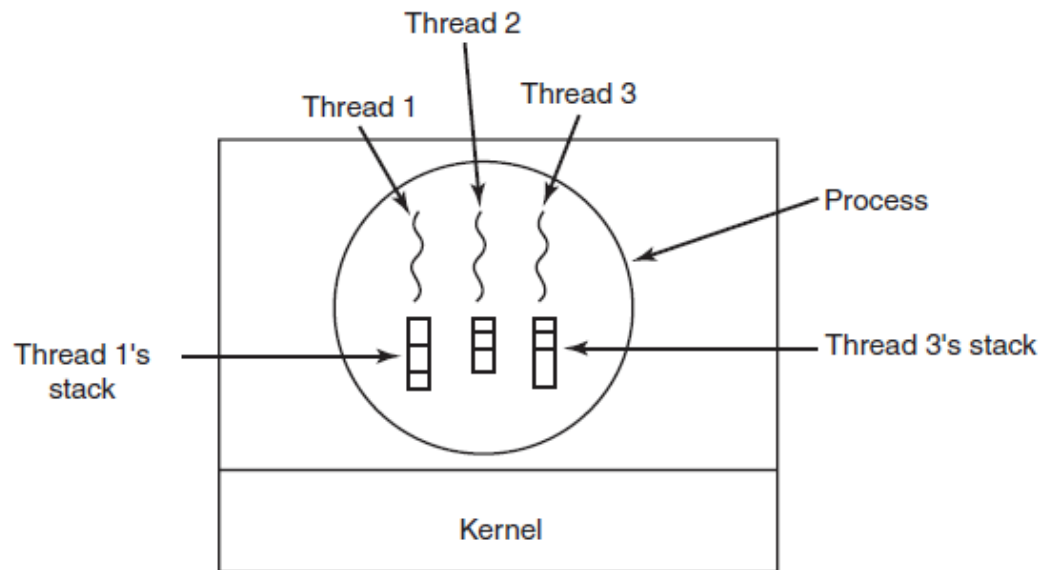


Figure 2-13. Each thread has its own stack.



Thread Libraries

- **Thread library** provides programmer with API for creating and managing threads
- Two primary ways of implementing
 - Library entirely in user space
 - Kernel-level library supported by the OS



POSIX Threads (Pthreads)

- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- ***Specification***, not ***implementation***
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Solaris, Linux, Mac OS X)



POSIX Threads in OS :

- The POSIX thread libraries are a C/C++ thread API based on standards. It enables the creation of a new concurrent process flow. It works well on multi-processor or multi-core systems, where the process flow may be scheduled to execute on another processor, increasing speed through parallel or distributed processing. Because the system does not create a new system, virtual memory space and environment for the process, threads need less overhead than “forking” or creating a new process. While multiprocessor systems are the most effective, benefits can also be obtained on uniprocessor systems that leverage delay in I/O and other system processes that may impede process execution.

POSIX Threads in OS :

- To utilise the PThread interfaces, we must include the header pthread.h at the start of the CPP script.

```
#include <pthread.h>
```

- PThreads is a highly concrete multithreading system that is the UNIX system's default standard.
- **PThreads** is an abbreviation for **POSIX threads**, and POSIX is an abbreviation for **Portable Operating System Interface**, which is a type of interface that the operating system must implement.
- PThreads in POSIX outline the **threading APIs** that the operating system must provide.



Why is Pthreads used?

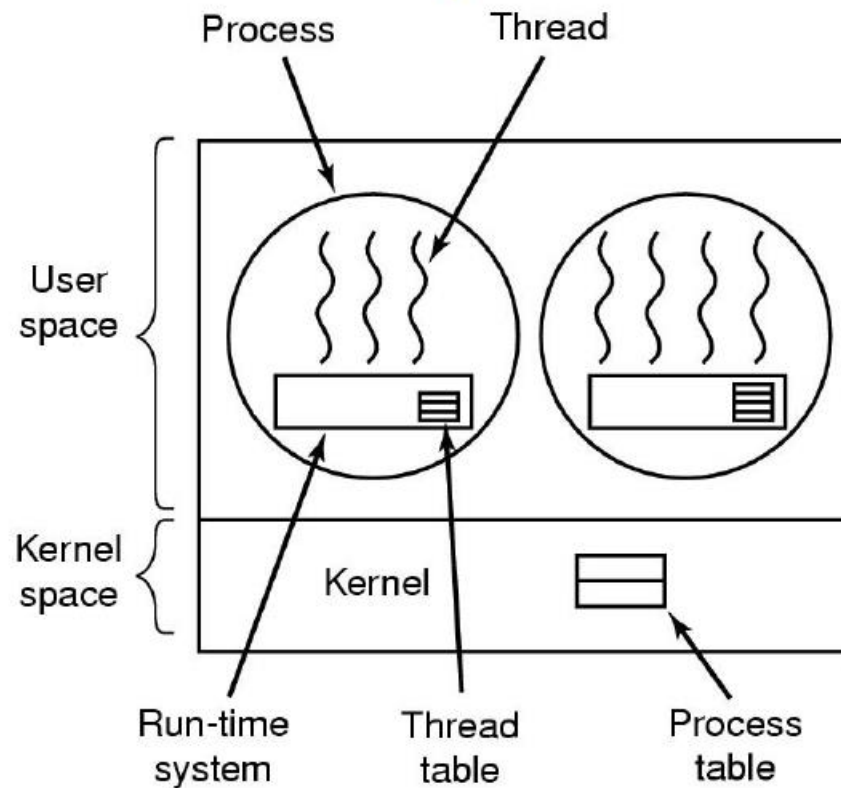
- The fundamental purpose for adopting **Pthreads** is to improve programme performance.
- When compared to the expense of starting and administering a process, a thread requires far less operating system overhead. Thread management takes fewer system resources than process management.
- A process's threads all share the same address space. Inter-thread communication is more efficient and, in many circumstances, more user-friendly than inter-process communication.
- Threaded applications provide possible performance increases and practical advantages over non-threaded programmes in a variety of ways.
- Multi-threaded programmes will run on a single-processor system but will automatically make use of a multiprocessor machine without the need for recompilation.
- The most significant reason for employing **Pthreads** in a multiprocessor system is to take advantage of possible parallelism.
- In order for a programme to use **Pthreads**, it must be divided into discrete, independent tasks that may run concurrently.



Implementation of Thread

- There are two ways to implement a thread, they're either in user space or in the Kernel.
- The corresponding code and the data structures used are stored in the user space. If an API is invoked, it results in a local system call in user space, rather than a system call.

Implementing Threads in User Space



A user-level threads package



Implementation of User-level Threads

Implementation at user-level

- User-level Thread Control Block (TCB), ready queue, blocked queue, and dispatcher.
- Kernel has no knowledge of the threads (it only sees a single process).
- If a thread blocks waiting for a resource held by another thread, its state is save and the dispatcher switches to another ready thread.
- Thread management (create, exit, yield, wait) are implemented in a runtime support library.



Pros - User-level Threads

- Pros
 - Thread management and switching at user level is much faster than doing it in kernel level
 - No need to trap into kernel and back to switch
 - Dispatcher algorithm can be tuned to the application
 - E.g. use priorities
 - Can be implemented on any OS (thread or non-thread aware)
 - Can easily support massive numbers of threads on a per-application basis
 - Use normal application virtual memory
 - Kernel memory more constrained. Difficult to efficiently support wildly differing numbers of threads for different applications.



Cons - User-level Threads

User-level Threads

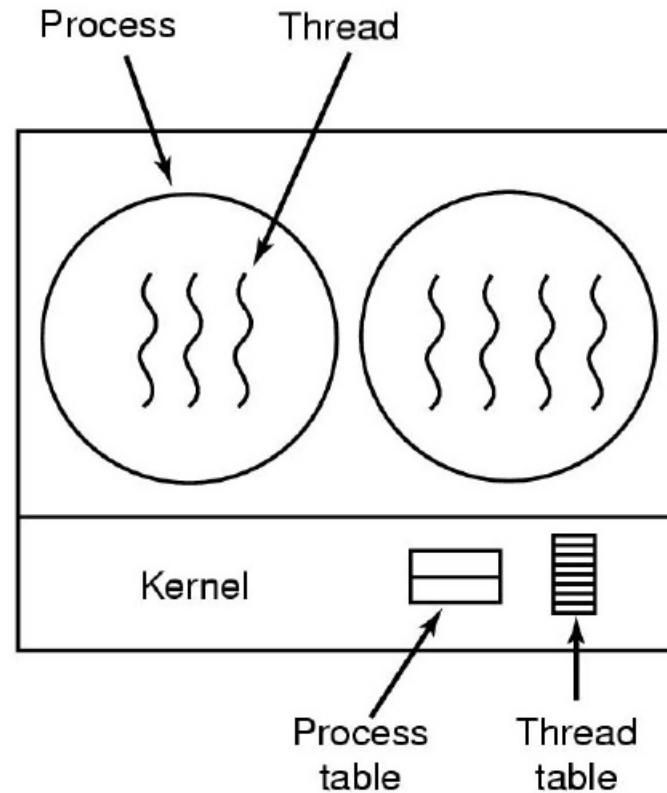
- Cons
 - Threads have to yield() manually (no timer interrupt delivery to user-level)
 - Co-operative multithreading
 - A single poorly design/implemented thread can monopolise the available CPU time
 - There are work-arounds (e.g. a timer signal per second to enable pre-emptive multithreading), they are course grain and a kludge.
 - Does not take advantage of multiple CPUs (in reality, we still have a single threaded process as far as the kernel is concerned)



Cons - User-level Threads

- Cons
 - If a thread makes a blocking system call (or takes a page fault), the process (and all the internal threads) blocks
 - Can't overlap I/O with computation
 - Can use wrappers as a work around
 - Example: wrap the `read()` call
 - Use `select()` to test if read system call would block
 - » `select()` then `read()`
 - » Only call `read()` if it won't block
 - » Otherwise schedule another thread
 - Wrapper requires 2 system calls instead of one
 - » Wrappers are needed for environments doing lots of blocking system calls?
 - Can change to kernel to support non-blocking system call
 - Lose “on any system” advantage, page faults still a problem.

Implementing Threads in the Kernel



A threads package managed by the kernel



Kernel Threads

- Cons
 - Thread creation and destruction, and blocking and unblocking threads requires kernel entry and exit.
 - More expensive than user-level equivalent
- Pros
 - **Preemptive** multithreading
 - Parallelism
 - Can overlap blocking I/O with computation
 - Can take advantage of a multiprocessor



Kernel Threads

- Cons
 - Thread creation and destruction, and blocking and unblocking threads requires kernel entry and exit.
 - More expensive than user-level equivalent
- Pros
 - **Preemptive** multithreading
 - Parallelism
 - Can overlap blocking I/O with computation
 - Can take advantage of a multiprocessor



Thread Switch

- A switch between threads can happen any time the OS is invoked
 - On a system call
 - Mandatory if system call blocks or on `exit()`;
 - On an exception
 - Mandatory if offender is killed
 - On an interrupt
 - Triggering a dispatch is the main purpose of the *timer interrupt*

A thread switch can happen between any two instructions

Note instructions do not equal program statements



Context Switch

- Thread switch must be *transparent* for threads
 - When dispatched again, thread should not notice that something else was running in the meantime (except for elapsed time)
- ⇒ OS must save all state that affects the thread
- This state is called the *thread context*
- Switching between threads consequently results in a *context switch*.

